It works, but if tomorrow we want to calculate dynamic pressure for ten different conditions, we end up repeating code.

Now, we want to reach this point:

q = calculate_dynamic_pressure(rho, speed)

In other words:

INPUTS → FUNCTION → OUTPUT

A function can be thought of as a small machine.

You provide it with:

$$\rho, V$$

The machine calculates:

$$q = \frac{1}{2}\rho V^2$$

And it returns the result to you.

`def`, parameters, and arguments

A function begins with:

def calculate_dynamic_pressure(rho, speed):

Here:

rho

speed

are **parameters**.

These are the names the function uses internally.

When we later call:

calculate_dynamic_pressure(1.225, 100.0)

then:

1.225

100.0

are **arguments**.

We can think of it as:

$$rho \leftarrow 1.225$$
$$speed \leftarrow 100$$

for that specific call.

`return` is VERY different from `print`.

This is arguably the most important concept in EX09.

We could write:

```
def calculate_dynamic_pressure(rho, speed):

    q = 0.5 * rho * speed**2

    print(q)
```

But that only **displays** the result.

On the other hand:

```
def calculate_dynamic_pressure(rho, speed):

    q = 0.5 * rho * speed**2

    return q
```

actually delivers the value to the rest of the program.

So, we can do this:

```
dynamic_pressure = calculate_dynamic_pressure(1.225, 100.0)
```

and now:

```
dynamic_pressure
```

holds the result, allowing us to:

print it

save it

compare it

write it to a file

use it in another equation

pass it to another function

That is far more powerful. **Scope also comes into play**.

Suppose:

```
def calculate_dynamic_pressure(rho, speed):


    q = 0.5 * rho * speed**2


    return q
```

The variable:

q

is created inside the function.

It is a local variable.

Then we do:

result = calculate_dynamic_pressure(1.225, 100.0)

and:

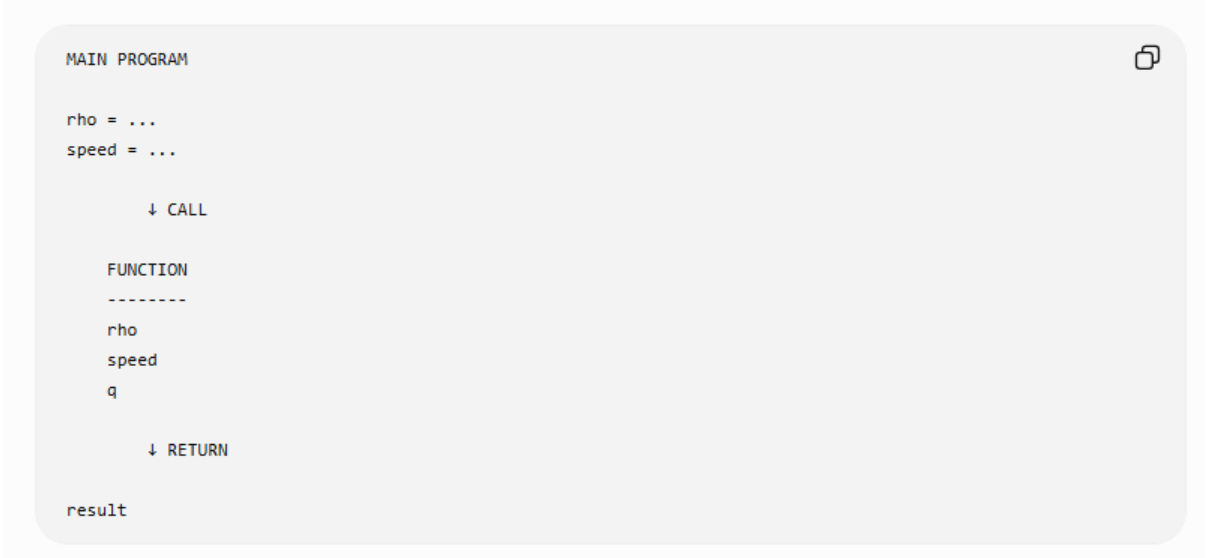
result

exists outside.

Think of it this way:

MAIN PROGRAM

This will be crucial as our programs grow.



Funciona, pero si mañana queremos calcular presión dinámica para diez condiciones distintas, empezamos a repetir código.

Ahora queremos llegar a:

q = calculate_dynamic_pressure(rho, speed)

Es decir:

INPUTS → FUNCTION → OUTPUT

Una función puede imaginarse como una pequeña máquina.

Le proporcionas:

$$\rho, V$$

la máquina calcula:

$$q = \frac{1}{2}\rho V^2$$

y te devuelve:

def, parámetros y argumentos

Una función empieza con:

def calculate_dynamic_pressure(rho, speed):

Aquí:

rho

speed

son **parámetros**.

Son los nombres que utiliza la función internamente.

Cuando posteriormente hagamos:

calculate_dynamic_pressure(1.225, 100.0)

entonces:

1.225

100.0

son **argumentos**.

Podemos pensarlo como:

$$rho \leftarrow 1.225$$
$$speed \leftarrow 100$$

durante esa llamada concreta.

---

return es MUY diferente de print

Esta será probablemente la idea más importante del EX09.

Podríamos hacer:

```
def calculate_dynamic_pressure(rho, speed):

    q = 0.5 * rho * speed**2

    print(q)
```

Pero eso únicamente **muestra** el resultado.

En cambio:

```
def calculate_dynamic_pressure(rho, speed):

    q = 0.5 * rho * speed**2

    return q
```

entrega realmente el valor al resto del programa.

Entonces podemos hacer:

```
dynamic_pressure = calculate_dynamic_pressure(1.225, 100.0)
```

y ahora:

```
dynamic_pressure
```

contiene el resultado y podemos:

imprimirlo

guardarlo

compararlo

escribirlo en un archivo

utilizarlo en otra ecuación

pasarlo a otra función

Eso es muchísimo más potente.

También aparece scope

Supón:

```
def calculate_dynamic_pressure(rho, speed):


    q = 0.5 * rho * speed**2


    return q
```

La variable:

q

se crea **dentro de la función**.

Es una variable local.

Después hacemos:

```
result = calculate_dynamic_pressure(1.225, 100.0)
```

y:

result

existe fuera.

Mentalmente:

MAIN PROGRAM

Esto será crucial cuando nuestros programas crezcan.

```
MAIN PROGRAM

rho = ...
speed = ...

        ↓ CALL

    FUNCTION
    --------
    rho
    speed
    q

        ↓ RETURN

result
```